

COMP 3311

DATABASE MANAGEMENT

SYSTEMS

LECTURE 10

STRUCTURED QUERY LANGUAGE (SQL):

DATA MANIPULATION LANGUAGE (DML)

ANALYTIC FUNCTIONS

- An analytic function computes an **aggregate value** for the query result, possibly based on a **grouping of tuples** of the query result called a **partition**.

✎ **Unlike an aggregate function, an analytic function can return multiple tuples for each group.**

- Within a partition, a **sliding window of tuples** can be used to **determine the range of tuples used** to perform the calculations for the **current tuple of the query result**.
- An analytic function is the last set of operations performed in a query except for the final **order by** clause.

✎ **Consequently, an analytic function can appear only in the **select clause** or final **order by clause**.**

Applications: sales-trend analysis, stock-market trend analysis, etc.

ANALYTIC VS. AGGREGATE FUNCTIONS

Query: For each account find the account information and the total balance for all accounts.

Account

<u>accountNo</u>	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place



Required Query Result

accountNo	balance	branchName	totalBalance
A-117	35000	Star House	290000
A-120	80000	Star House	290000
A-215	55000	Diamond Hill	290000
A-301	75000	Pacific Place	290000
A-322	45000	Pacific Place	290000

Is this correct?

No! Why?

~~**select** accountNo, balance, branchName, **sum**(balance) **as** totalBalance
from Account;~~

Is this correct?

No! Why?

~~**select** accountNo, balance, branchName, **sum**(balance) **as** totalBalance
from Account
group by accountNo, balance, branchName;~~

select accountNo, balance, branchName,
 sum(balance) **over** () **as** totalBalance
from Account;

This query calculates **sum**(balance) over the entire query result and then includes it in each query result tuple.

ANALYTIC VS. AGGREGATE FUNCTIONS (CONT'D)

Query: For each account find the account information and the total balance for the account's branch.

Account

<u>accountNo</u>	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place



Required Query Result

accountNo	balance	branchName	totalBranchBalance
A-117	35000	Star House	115000
A-120	80000	Star House	115000
A-215	55000	Diamond Hill	55000
A-301	75000	Pacific Place	120000
A-322	45000	Pacific Place	120000

Is this correct?
No! Why?

~~`select accountNo, balance, branchName, sum(balance) as totalBranchBalance
from Account
group by branchName;`~~

`select accountNo, balance, branchName,
sum(balance) over (partition by branchName) as totalBranchBalance
from Account;`

This query first calculates `sum(balance) over` the `partitions of branchName` of the query result (i.e., for each branch) and then includes it in each query result tuple.

ORACLE ANALYTIC FUNCTION SYNTAX

analytic_function([arguments])

over ([**partition by** clause] [**order by** clause [*windowing_clause*]])

analytic_function – a function that **computes an aggregate value over a group of tuples** (e.g., avg, sum, rank, percentile rank, nth value, linear regression, etc.).

partition by clause – **optional**; partitions the query result into groups based on one or more criteria (like a **group by** clause). If this clause is omitted, then the function treats all tuples of the query result as a single group.

```
select accountNo, balance, branchName,  
       sum(balance) over (partition by branchName) as totalBranchBalance  
from Account;
```

order by clause – **optional**; specifies *how data is ordered within a partition*.

```
select accountNo, balance, branchName,  
       sum(balance) over (partition by branchName order by accountNo)  
       as totalBranchBalance  
from Account;
```

👉 **How does a partition by clause differ from a group by clause?**

ORACLE ANALYTIC FUNCTIONS

Analytic functions followed by an asterisk (*) allow the full syntax, including the *windowing_clause* (discussed on later slides).

AVG *

CORR *

COUNT *

COVAR_POP *

COVAR_SAMP *

CUME_DIST

DENSE_RANK

FIRST

FIRST_VALUE *

LAG

LAST

LAST_VALUE *

LEAD

LISTAGG

MAX *

MEDIAN

MIN *

NTH_VALUE *

NTILE

PERCENT_RANK

PERCENTILE_CONT

PERCENTILE_DISC

RANK

RATIO_TO_REPORT

REGR_ (Linear Regression) Functions *

ROW_NUMBER

STDDEV *

STDDEV_POP *

STDDEV_SAMP *

SUM *

VAR_POP *

VAR_SAMP *

VARIANCE *

See Oracle Database [Data Warehousing Guide](#) for more information on these functions and for scenarios illustrating their use.

RANK FUNCTION

- The **rank** function allows the **position of a value within a set of values** to be determined.

Query: For each bank branch find its name, assets and rank with respect to the assets of all branches.

```
select branchName, assets,  
       rank() over (order by assets desc) as branchRank  
from Branch;
```

Query result

branchName	assets	branchRank
Star House	115000	1
Festival Walk	107000	2
Pacific Place	40000	3
Diamond Hill	0	4

 **Ranking requires an order by specification.**

RANK FUNCTION (CONTD)

- Since query result tuples are unordered, if ordering by rank is required the query result further needs to be sorted by rank requiring an additional **order by** clause.

Query: For each bank branch find its name, assets and rank with respect to the assets of all branches. Order the result by rank descending.

```
select branchName, assets,  
       rank() over (order by assets desc) as branchRank  
from Branch  
order by branchRank desc;
```

Query result

branchName	assets	branchRank
Diamond Hill	0	4
Pacific Place	40000	3
Festival Walk	107000	2
Star House	115000	1

RANK FUNCTION (CONTD)

- The **rank** function can be expressed using the basic **SQL** aggregation functions.

Query: For each bank branch find its name, assets and rank with respect to the assets of all branches. Order the result by rank descending.

```
select branchName, assets, (1 + (select count(*)  
                                from Branch B  
                                where B.assets>A.assets)) as branchRank  
from Branch A  
order by branchRank desc;
```

✎ Such queries may be difficult to express and inefficient to evaluate.

✎ The **rank** function makes such queries much easier to express and more efficient to evaluate.

RANK FUNCTION: PARTITIONS

- The **rank** function can be applied separately to each partition created by a **partition by** clause.

Query: For each account find its branch name, account number, balance and branch rank with respect to all account balances. Order the result by branch name and rank.

```
select branchName, accountNo, balance,  
       rank() over (partition by branchName  
                    order by balance desc) as branchAccountRank  
from Account  
order by branchName, branchAccountRank;
```

Accounts are partitioned by branch name and ranked separately within each partition.

Account

<u>accountNo</u>	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place



Query result

branchName	accountNo	balance	branchAccountRank
Diamond Hill	A-215	55000	1
Pacific Place	A-301	75000	1
Pacific Place	A-322	45000	2
Star House	A-120	80000	1
Star House	A-117	3500	2

RANK FUNCTION: TOP/BOTTOM N TUPLES

- To return only the top-ranking tuples in a query result, the ranking query can be **nested within a containing query** whose **where** clause chooses only those tuples whose rank is lower than a specified value.

Query: Find the name, assets and rank of the top three ranked branches with respect to the assets of all branches.

```
select branchName, assets
from (select branchName, assets,
      rank() over (order by assets desc) as branchRank
      from Branch)
where branchRank<=3;
```

Query result

branchName	assets	branchRank
Star House	115000	1
Festival Walk	107000	2
Pacific Place	40000	3

👉 Multiple tuples with the same rank (i.e., ties) are included in the result.

👉 Bottom *N* ranking is the same as top *N* ranking but with a reverse sorting order.

RANK FUNCTION: NOTES

- Null values are normally treated as the highest values.
 - Can be changed using **nulls first** or **nulls last** keywords.
- Multiple **rank** expressions can be used within a single **select** statement (e.g., overall rank and rank within a department).
- When ranking occurs along with a **group by** clause, the **group by** clause is applied first and ranking is done on the results of the **group by**.
- Several other functions can be used in place of **rank**.
 - **precent_rank** – gives the rank of a tuple as a fraction (defined as $(r-1)/(n-1)$ where n is the number of tuples and r is the rank of the tuple).
 - **cume_dist** – gives the cumulative distribution for a tuple (defined as p/n where p is the number of tuples with ordering values preceding or equal to the ordering value of the tuple and n is the number of tuples).
 - **row-number** – sorts the rows and gives each row a unique row number based on its position in the ordering; rows with the same ordering value get different row numbers in a nondeterministic fashion.

DENSE_RANK FUNCTION

- The **rank** function gives the same rank to all tuples that have equal value on the **order by** attributes.
 - E.g., if two branches have the same highest asset value, both would get rank 1.
- This results in "gaps" in the ranking
 - E.g., if two branches have the top rank of 1, then the next rank would be 3, not 2.
- The **dense_rank** function eliminates gaps in the ordering; ties still get the same rank.

DENSE_RANK FUNCTION (CONTD)

Query: For each branch find its name, assets and branch rank with respect to the assets of all branches.

```
select branchName, assets  
       rank() over (order by assets desc) as branchRank  
from Branch  
order by branchRank;
```

Query result

branchName	assets	branchRank
Star House	115000	1
Festival Walk	107000	2
Langham Place	107000	2
Pacific Place	40000	4
Diamond Hill	0	5

← “gap” in ranking

rank function result

Branch

<u>branchName</u>	district	liabilities	assets
Diamond Hill	Kwun Tong	55000	0
Festival Walk	Yau Tsim Mong	0	107000
Pacific Place	Central and Western	120000	40000
Star House	Yau Tsim Mong	115000	115000
Langham Place	Yau Tsim Mong	100000	107000

DENSE_RANK FUNCTION (CONTD)

Query: For each branch find its name, assets and branch rank, without gaps, with respect to the assets of all branches.

```
select branchName, assets  
       dense_rank() over (order by assets desc) as branchRank  
from Branch  
order by branchRank;
```

Query result

branchName	assets	branchRank
Star House	115000	1
Festival Walk	107000	2
Langham Place	107000	2
Pacific Place	40000	3
Diamond Hill	0	4

← no “gap” in ranking

dense_rank function result

Branch

<u>branchName</u>	district	liabilities	assets
Diamond Hill	Kwun Tong	55000	0
Festival Walk	Yau Tsim Mong	0	107000
Pacific Place	Central and Western	120000	40000
Star House	Yau Tsim Mong	115000	115000
Langham Place	Yau Tsim Mong	100000	107000

NTILE FUNCTION

- A common ranking type of query is to **determine the percentile in which a value in a multi(set) belongs** (e.g., top, bottom half; top, middle or bottom third; first, second, third or fourth quartile; etc.).
- The $\text{ntile}(n)$ function takes the tuples in the specified order and divides them into n buckets with an equal number of tuples*.
- For each tuple, it then appends the number of the bucket into which the tuple is placed, with bucket numbers starting with 1.

 **Particularly useful for constructing histograms based on percentiles.**
(Which will be discussed in a later lecture.)

* If the total number of tuples is not divisible by n , then the number of tuples in each bucket can differ by at most 1. Tuples with the same value for the ordering attribute may be assigned to different buckets nondeterministically, in order to make the number of tuples in each bucket equal.

NTILE FUNCTION (CONTD)

Query: For each branch find its name, assets and the quartile into which it falls with respect to the assets of all branches.

```
select branchName, assets,  
       ntile(4) over (order by assets desc) as branchQuartile  
from Branch;
```

Query result

branchName	assets	branchQuartile
Star House	115000	1
Festival Walk	107000	1
Langham Place	107000	2
Pacific Place	40000	3
Diamond Hill	0	4

LISTAGG FUNCTION

- The **listagg** function concatenates data from multiple tuples into a single list of values separated by a specified delimiter.

Query: Find the districts of clients and return them as a comma-separated list.

```
select listagg(district, ', ') as "Client districts"  
from Client;
```

Query result

Client districts
Central and Western, Islands

- Duplicate values in a list can be removed using the **distinct** keyword and the list elements can be sorted using the **within group** clause along with an **order by** clause.

Query: Find the unique districts of clients and return them as a comma-separated list in ascending order.

```
select listagg(distinct district, ', ') within group (order by district)  
from Client;
```

LISTAGG FUNCTION (CONT'D)

- When used with a **group by** clause, the **listagg** function **returns a list of values for each group**.

👉 **Useful for aggregating data from several tuples into a single tuple associated with a specific value.**

Query: Find the names of the clients in each district and return them as a comma-separated list. Order the query result by district ascending.

Is this correct?
No!
Why?

~~**select** district, name
from Client
group by district
order by district, name;~~

Is this correct?
Yes,
but ...

~~**select** district, name
from Client
group by district, name
order by district, name;~~

Query result

district	name
Central and Western	John Lee
Central and Western	May Cheung
Central and Western	Steven Chan
Islands	Mary Kwan
Southern	Andy Lau
Southern	Paul Cha
null	Isaac Li
null	Pam Du

The district name repeats for each client.

The client names are not in a list.

LISTAGG FUNCTION (CONT'D)

Query: Find the names of the clients in each district and return them as a comma-separated list in ascending order. Order the query result by district ascending.

```
select district, listagg(name, ', ') within group (order by name) as clients
from Client
group by district
order by district;
```

Query result

district	clients
Central and Western	John Lee, May Cheung, Steven Chan
Islands	Mary Kwan
Southern	Andy Lau, Paul Chan
null	Isaac Li, Pam Du

LISTAGG FUNCTION (CONT'D)

- When used as an analytic function, **listagg** requires an **over** clause and **returns a list of values for each partition**.

Query: For each loan in 2025 find the loan number, loan amount, branch name and a comma-separated list in ascending order of all loan amounts given in the same year at the same branch. Order the query result by the loan amount ascending.

```
select loanNo, amount, branchName,  
       listagg(amount, ', ' within group (order by amount)  
       over (partition by branchName) as "all loan amounts"  
from Loan  
where year='2025'  
order by amount;
```

✋ **An order by clause is not allowed in the over clause for the listagg function.**

Query result

loanNo	amount	branchName	all loan amounts
L-434	52000	Festival Walk	52000, 55000
L-435	55000	Festival Walk	52000, 55000

✋ **Cannot use a group by clause to get this query result.**
Why?

LISTAGG FUNCTION SYNTAX AND NOTES

```
listagg ([all | distinct] column_name [, delimiter])  
[within group(order_by_clause)]  
[over(query_partition_clause)]
```

all | distinct – **all** (the default) retains all values; **distinct** removes duplicate values.

column_name – a column, constant, or expression that involves the columns.

delimiter – a string that separates the result values. If **delimiter** is omitted, the function uses a **null** value (i.e., no delimiter).

order_by_clause – a list of sort expressions to sort the list values in ascending (**asc**) or descending (**desc**) order. The **nulls first** or **nulls last** option can be used to control the sort order of nulls. The defaults are **ascending** order and **nulls last**.

As a **single-set aggregate function**, **listagg** operates on all result tuples and returns a concatenated string containing all **column_name** values as a single tuple.

As a **group-set aggregate function**, **listagg** operates on and returns an output tuple for each group defined by the **group by** clause.

As an **analytic function**, **listagg** partitions the query result into groups based on the **query_partition_clause**.

WINDOWING

analytic_function([arguments])
 over ([**partition by** clause] [**order by** clause [*windowing_clause*]])

- A **windowing** clause allows an analytic function to be **computed over a moving partition** called a **window**.

 **Useful for computing an aggregate over a fixed time range.**

- The **window moves** (slides) **over the tuples** of a query result, and, for each tuple, the window is used to calculate the function result.
- A **start point**, **end point** or **both for the window** can be specified either in terms of the number of tuples in the window (a physical set) or in terms of a range (a logical offset).
- Unlike partitions, **windows may overlap** so that a tuple may contribute to multiple windows.

WINDOWING (CONTD)

Query: For each year find the average total loan amounts for the year as well as the previous two-year period.

loanNo	amount	year	branchName
L-170	30000	2021	Star House
L-320	20000	2021	Pacific Place
L-172	30000	2022	Star House
L-215	50000	2022	Diamond Hill
L-322	15000	2022	Pacific Place
L-323	40000	2022	Pacific Place
L-333	50000	2022	Pacific Place
L-174	90000	2023	Star House
L-210	45000	2023	Diamond Hill
L-556	58000	2023	Langham Place
L-171	115000	2024	Sar House
L-211	35000	2024	Diamond Hill
L-212	8000	2024	Diamond Hill
L-213	45000	2024	Diamond Hill
L-321	40000	2024	Pacific Place
L-436	10000	2024	Festival Walk
L-555	17000	2024	Langham Place
L-214	5000	2025	Diamond Hill
L-434	52000	2025	Festival Walk
L-435	55000	2025	Festival Walk

```
select year,
       avg(yearLoanTotal)
       over (order by year rows 2 preceding)
       as movingAvgYearLoanTotal
from (select year, sum(amount) as yearLoanTotal
      from Loan
      group by year);
```

Subquery result

year	yearLoanTotal
2021	50000
2022	185000
2023	193000
2024	270000
2025	112000



Query result

year	movingAvgYearLoanTotal
2021	50000
2022	117500
2023	142666.7
2024	216000
2025	191666.7

The query window includes the current tuple and the two preceding (previous) tuples, if any.
The moving average is calculated over the tuples in the current window.

Loan(loanNo, amount, year, branchName)

WINDOWING (CONTD)

- A **window** can include tuples both **preceding** and **following** the current tuple.
- It also can be **unbounded** (preceding or following) meaning that all tuples preceding or all tuples following the current tuple are included in the window.

WINDOWING (CONTD)

Query: For each year find the average total loan amounts for the year, for all previous years as well as for the following year.

loanNo	amount	year	branchName
L-170	30000	2021	Star House
L-320	20000	2021	Pacific Place
L-172	30000	2022	Star House
L-215	50000	2022	Diamond Hill
L-322	15000	2022	Pacific Place
L-323	40000	2022	Pacific Place
L-333	50000	2022	Pacific Place
L-174	90000	2023	Star House
L-210	45000	2023	Diamond Hill
L-556	58000	2023	Langham Place
L-171	115000	2024	Sar House
L-211	35000	2024	Diamond Hill
L-212	8000	2024	Diamond Hill
L-213	45000	2024	Diamond Hill
L-321	40000	2024	Pacific Place
L-436	10000	2024	Festival Walk
L-555	17000	2024	Langham Place
L-214	5000	2025	Diamond Hill
L-434	52000	2025	Festival Walk
L-435	55000	2025	Festival Walk

```
select year,
       avg(yearLoanTotal) over (order by year rows
                               between unbounded preceding and 1 following)
       as movingAvgYearLoanTotal
from (select year, sum(amount) as yearLoanTotal
      from Loan
      group by year);
```

Subquery result

year	yearLoanTotal
2021	50000
2022	185000
2023	193000
2024	270000
2025	112000



Query result

year	movingAvgYearLoanTotal
2021	117500
2022	142666.7
2023	174500
2024	16200
2025	16200

The query window includes the current tuple, all preceding (previous) tuples and the following tuple, if any.
The moving average is calculated over the tuples in the current window.

Loan(loanNo, amount, year, branchName)

WINDOWING (CONTD)

- A **window** can be defined for **partitions**.
- The keyword **range** can be used in place of **rows** which allows the window to cover all tuples with a particular value.

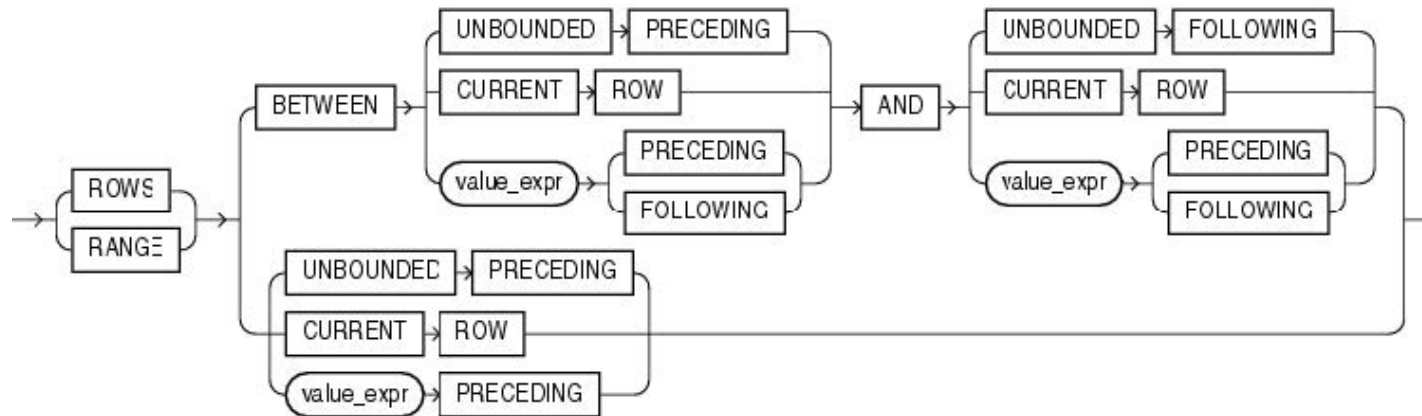
Query: Find the average total loan amounts for each branch for each year.

```
select distinct branchName, year,  
    avg(amount) over (partition by branchName order by year  
    range current row) as avgYearLoanTotal  
from Loan;
```

Since the query window ranges over each (branchName, year) combination, the preceding query is equivalent to:

```
select branchName, year, avg(amount) as avgYearLoanTotal  
from Loan  
group by branchName, year;
```

ORACLE WINDOWING_CLAUSE SYNTAX



rows | range – defines for each row in a partition a window either of rows (i.e., a physical unit) or a range (i.e., a logical offset) used for calculating the function result. The function is then applied to all the rows in the window which moves through the partition from top to bottom.

between ... and – specifies a start point (expression before **and**) and end point (expression after **and**) for the window.

unbounded preceding – indicates that the window starts at the first row of the partition.

unbounded following – indicates that the window ends at the last row of the partition.

current row – specifies that the window begins at the current row (if rows is specified) or value (if range is specified).

TUPLE INSERTION

- The **insert** command adds one or more tuples to a relation.

Example: Add a new account.

```
insert into Account values ('A-332', 1200, 'Pacific Place');
```

Example: Add a new account with balance set to null (if allowed).

```
insert into Account values ('A-333', null, 'Pacific Place');
```

✎ The order of the values must match the order of the attributes in the relation.

- Attribute names need to be *specified explicitly* for order-independent insertion and to make use of default values.

```
insert into Account (accountNo, branchName, balance) values ('A-334', 'Pacific Place', 1200);
```

```
insert into Account (accountNo, branchName) values ('A-335', 'Pacific Place');
```

Omits the balance value.

Account(accountNo, balance, branchName)



COMPLEX INSERTION

- Insertion values can be obtained from a query result.

Example: Create a \$200 savings account for all loan clients of the Pacific Place branch. Let the loan number serve as the account number for the new savings account.

```
insert into Account  
(select loanNo, 200, branchName  
from Loan  
where branchName='Pacific Place');
```

```
insert into Depositor  
(select clientId, loanNo  
from Loan natural join Borrower  
where branchName='Pacific Place');
```

The order and type of the attributes in the **select** clause must match the order and type of the attributes in the relation being inserted into.

Note: The keyword **values** is omitted when the values are obtained from a **select** statement.

Account(accountNo, balance, *branchName*)

Loan(loanNo, amount, year, *branchName*)

COMP 3311



Depositor(clientId, accountNo)

Borrower(clientId, loanNo)

LECTURE 10
SQL DML

TUPLE DELETION

- The **delete** command deletes *zero or more tuples* from a relation where, possibly, the tuples satisfy some condition.

Example: Delete all accounts at the Pacific Place branch.

```
delete from Account  
where branchName='Pacific Place';
```

Conceptually, deletion is done in two steps.

1. Find the tuples to delete.
2. Delete the tuples found.

 **Can only delete if no integrity constraints are violated!**

- What do you think this statement does?

```
delete from Account;
```

COMPLEX DELETION

- The **where** clause predicate in a **delete** statement can be as complex as in a **select** statement.

Example: Delete all accounts at the Star House branch.

```
delete from Depositor
where accountNo in (select accountNo
                    from Depositor natural join Account
                    where branchName= 'Star House');
```

➡ Must also delete the accounts of these depositors!

➡ Can the deletion of the accounts be specified using a referential integrity constraint?

TUPLE UPDATE

- The **update** command is used to change a value in a tuple.

Example: Set the balance of account A-333 to \$50,000.

```
update Account set balance=50000 where accountNo= 'A-333';
```

Example: Increase the balance of all accounts whose balance is less than \$10,000 by 5%.

```
update Account set balance=balance*1.05 where balance<10000;
```

- What do you think this statement will do?

```
update Account set balance=balance+100;
```

COMPLEX UPDATE

- Updates can be specified using a **case** statement.

Example: Increase all accounts with balance over \$10,000 by 6%;
all other accounts receive 5%.

```
update Account
set balance=balance*1.06
where balance>10000;
```

```
update Account
set balance=balance*1.05
where balance<=10000;
```

 **Need two update statements! The order is important! Why?**

- These two updates can be stated in one statement using a **case** statement.

```
update Account
set balance=case
    when balance<=10000 then balance*1.05
    else balance*1.06
end;
```